Proving Cedar Authorization Decisions:

A zkVM, a Policy-to-Circuit Compiler, and a Machine-Checked Proof System

Conner Swann

Preprint, 6 November 2026

Abstract

A resource server normally trusts an authorization decision because it trusts whoever made it. Proof-carrying authorization replaces that trust with a proof that the decision follows from the policy and the facts. We measure what this costs for Cedar, a widely used policy language with a formal model in Lean, on three kinds of proof system that read the same committed entity records: the unmodified Cedar evaluator in the RISC Zero zkVM; Noir circuits produced by a new compiler that consumes the typed terms of Cedar's verified symbolic compiler; and the same compiled form on GordieHowe, a proof system whose soundness and zero knowledge are proved in Lean. On a laptop, one TinyTodo decision takes 768 s to prove in the zkVM, 2.08 s with Noir and 43 ms with GordieHowe, with proofs of 2.24 MB, 14.7 KB and 125 KB that verify in 15 to 126 ms. On every backend the policy is the cheapest part of the proof. The zkVM spends its cycles parsing, with policy evaluation at 0.3 percent of cycles over a full entity store, and the circuits spend their gates on SHA-256. The compiler handles 5 of 7 sample policy sets, agrees with Cedar on all 40 of their test requests, and rejected every decision-changing attack in an adversarial test suite. Compiled circuits prove in 4.5 to 7.9 s in a browser, and signing the facts adds 47 thousand gates with P-256 or 434 thousand with Ed25519. Our translation is not verified, zero knowledge is not established for the programs we measured, and the per-program Lean certificate is incomplete.

1 Introduction

When a resource server asks a policy decision point (PDP) whether a request is allowed, it accepts the answer on trust. It trusts the PDP's operator, the key that signs the answer, and the channel that carries it. Within a single organization this is a reasonable arrangement. It breaks down when the party making the decision is not one the resource server would trust: a client device, the runtime of an AI agent acting on someone's behalf, a vendor's service, or a machine in someone else's building. Signing the answer does not help, because a signature establishes who answered, not that the answer is correct.

The alternative is old. Appel and Felten proposed in 1999 that clients submit proofs of authorization for the server to check [1]. Bauer and colleagues built a web system around a small proof checker at the server [2], SD3 had its evaluator emit a proof alongside each answer [13], and Evidence-Based Audit kept such proofs as receipts [26]. These systems proved authorization from premises the verifier could see. Modern proof systems add two things: proofs are succinct, and their premises can stay hidden. Prezta uses this to prove XACML decisions in the RISC Zero zkVM for edge devices [28], and it is the closest prior work to ours.

We ask a narrow question. Take one real policy language with a formal semantics and run it on each kind of proof system available today. What does one decision cost, where does the cost come from, and what does each system guarantee? Cedar suits the question unusually well. It has a Lean model, a Rust evaluator tested differentially against that model [8], and a symbolic compiler, verified in Lean, that reduces each policy to a typed term [25]. We carry the proof inside an ordinary AuthZEN response [10], so the wire protocol does not change.

We build three backends that read the same committed entity records. The first runs the published Cedar evaluator, unmodified, inside a zkVM; it supports all of Cedar and treats the policy as an input. The second is a compiler from validated Cedar to Noir circuits that, instead of its own front end, consumes the terms Cedar's symbolic compiler already produces. The third sends the same compiled form to GordieHowe [23], a proof system whose knowledge soundness and zero knowledge are machine-checked in Lean.

The measurements tell a consistent story. Proving one decision takes 768 s in the zkVM, 2.08 s with Noir and 43 ms with GordieHowe, and on every backend the policy itself is the cheapest part. The zkVM spends its cycles parsing text into Cedar's data structures, and the circuits spend their gates on SHA-256. Signatures follow the same pattern: they cost far more than the policy but little against the rest of the proof. In a browser, the compiled Noir circuits prove in 4.5 to 7.9 s.

Our contributions are:

1. A compiler from Cedar to arithmetic circuits built on the term language of Cedar's verified symbolic compiler, targeting both Noir and GordieHowe (Section 5.2).
2. A measurement of one policy language on a zkVM, on compiled circuits and on a machine-checked proof system, all reading the same committed records (Section 6).
3. Evidence that parsing, not policy evaluation, dominates the cost of a policy engine in a zkVM (Section 6.2).
4. A fixed-shape record commitment that stores each entity's ancestors in its leaf, which makes a proof of denial cost the same as a proof of permission (Section 4.2).
5. An adversarial test suite for generated circuits (Section 7.1).
6. The cost of signed facts with P-256 and with Ed25519, including an Ed25519 verifier for Noir written from RFC 8032 (Sections 4.3 and 6.4).

When proofs are unnecessary. Inside one organization with a trusted PDP, a signed answer does the same job at far lower cost. Where the verifier may hold the policy and the facts, it should simply decide again: Cedar evaluates in microseconds [6], while our proofs take milliseconds just to verify. In a well-run data center, a trusted execution environment is cheaper still and covers the whole language [14]. Proofs earn their cost in two settings: where the party deciding controls the hardware, now that physical interposers can extract attestation keys [5], and where the facts behind a decision must stay hidden from the party checking it.

2 Background

Cedar. A Cedar policy set maps a request and an entity store to a decision. A request names a principal, an action, a resource and a context, and the decision is *allow* exactly when at least

one `permit` policy is satisfied and no `forbid` policy is. Entities carry attributes and parents, and the operator `in` tests membership in the transitive closure of the parent relation. Cedar's semantics is formalized in Lean, and the production Rust evaluator is tested differentially against that model [8]. The parsers are outside the model.

Symbolic compilation. Because Cedar validates policies against a schema, every expression has a known type once the principal type, action and resource type are fixed. Cedar's symbolic compiler, `cedar-policy-symcc`, uses this to reduce each policy, for each such combination, to a typed term over the request and the entity store. These terms are normally handed to an SMT solver for policy analysis. SymCert verifies the compiler in Lean [25]. Its correctness theorems concern the evaluation of a term under an interpretation and do not depend on the solver, which is what lets the same terms serve as input to a circuit compiler.

AuthZEN. The OpenID AuthZEN Authorization API 1.0 [10] standardizes the exchange between an enforcement point and a PDP. A request carries a subject, an action, a resource and a context, and a response carries a decision with an optional `context` object. Receivers ignore fields they do not recognize, although Section 5.5 of the specification allows an enforcement point to reject a permit that arrives with context it does not understand. A proof can therefore travel in the response without any change to the specification.

Proof systems. The three backends sit at different points of a familiar trade-off. A zkVM proves that a program for a general-purpose instruction set ran correctly on some input. Anything that compiles for the target can be proved, at a cost per executed cycle; we use RISC Zero [4]. A circuit-based system proves that a fixed arithmetic circuit is satisfied. Each operation is far cheaper, but everything must be expressed as constraints; we use Noir 1.0.0 with Barretenberg 6.0.0's UltraHonk prover. GordieHowe [23] is a language, a certifying compiler and a proof system together. Its knowledge soundness and zero knowledge are proved in Lean, and the theorems apply to a particular program once two conditions hold: the source program is complete and round-by-round knowledge sound, and the compiler's per-program certificate is accepted by Lean's kernel.

3 Setting and threat model

Five parties are involved. A *policy authority* writes the policy set and publishes its hash and schema. An *attribute authority* commits to, or signs, the entity data; it alone computes the transitive closure of parents. The *decider* evaluates the policy and produces the proof, the *verifier* checks the proof and enforces the decision, and an *auditor* may re-check stored responses later.

For integrity, the decider is the adversary. It may try to obtain a permit the policy denies, replay an old permit, alter facts, substitute a different policy or evaluator, or attach a valid proof to a request other than the one being enforced. To prevent this, the verifier pins an evaluator identity, a policy hash, and either an entity root or an attribute authority's key, and every proof binds a hash of the request that includes a nonce chosen by the verifier. For privacy, the roles reverse: the verifier is the adversary, and in the private mode it should learn nothing about the facts beyond the decision. Section 7.2 discusses which backends achieve this.

Four threats are outside this work. A publisher could show different roots to different verifiers, the problem key transparency solves [18]. A proof could rely on a stale root, the "new enemy" problem of Zanzibar [21]. A verifier free to choose policies learns one bit per query, so a decision is an oracle over private facts. And we treat the request itself as public, so nothing here is anonymous. Availability, side channels on the decider's device, and policies that are simply wrong are also out of scope.

4 Design

4.1 The proof-carrying response

The request is an unmodified AuthZEN evaluation request, to whose `context` the verifier may add a nonce or an epoch. The response is an AuthZEN decision whose `context` contains a proof object: a format identifier, the evaluator's identity, the public values, and the proof itself. All three backends expose the same kinds of public value: a hash of the request including its nonce; hashes of the policy set and schema; the entity root, or in the signed mode the authority's key and epoch; and the decision. The circuit backends add a hash of the request context, for reasons given in Section 4.2.

The values are not byte-for-byte identical across backends. The zkVM, which parses JSON natively, hashes the entire request, context included, in canonical JSON form (RFC 8785). The circuit backends hash a fixed binary layout of the request with the context reduced to the nonce, and bind the context through its own hash. The format identifier tells the verifier which convention applies.

Verification is mechanical. The verifier checks the proof against the pinned evaluator, recomputes the request hash from the request it is about to enforce, compares the remaining values against those it accepts, and enforces the decision contained in the proof. A proof that verifies is not a permit; a verified deny is a deny. Cedar can also report which policies determined a decision, but that list can reveal attributes, so the private mode omits it.

4.2 Committing to facts

A decider that could choose its own facts could prove anything, so every proof opens the facts it uses from a commitment the verifier already holds. The shape of that commitment largely determines the cost of a proof, and we designed it around what a circuit does cheaply.

Each entity becomes a fixed-shape record: a hash of its identifier, a presence byte, eight attribute slots in schema order, and sixteen ancestor slots. The ancestor slots hold the *transitive closure* of the entity's parents, computed once by the attribute authority rather than by the decider. A binary SHA-256 Merkle tree over all records is the store's commitment. Sets and nested records are stored behind a hash and opened only when a policy reads them.

Placing the closure in the leaf makes denials cheap. Checking whether a principal is `in` a group, or is *not* in it, becomes a scan of known slots in a record the proof has already opened. A proof of denial needs no sorted tree and no proof of absence, and costs the same as a proof of permission. For TinyTodo, each decision opens exactly two records, the principal's and the resource's, and the size of the store does not affect the circuit up to the depth of the tree. The price is a set of hard limits. An entity with more than 16 ancestors or more than 8 attributes cannot be committed, and a principal or resource missing from the store cannot be opened at

all. Cedar treats such an entity as having no attributes and no parents; our circuits produce no proof.

Two further choices keep circuits from reconstructing text. First, because the verifier computes the request hash itself, we are free to choose what it hashes. Each string is encoded as a length byte followed by its bytes, zero-padded to a fixed capacity: 24 bytes for an entity type, 48 for an identifier, and 32 each for an action and the nonce. Longer strings are refused. Both the identifier hash and the request hash use this layout, so a circuit never rebuilds JSON or Cedar's textual form of an identifier, and one circuit serves every string length. Because the padding lies inside the hash, an alternative encoding of the same request yields a hash no verifier will accept. Second, the request context, whose JSON would be large and fragile to rebuild in a circuit, is encoded as one more record in the layout the schema gives for the request's types, and bound to its own public hash. The verifier already holds the context, so computing that hash requires no new trust. It costs one record digest, about 20 thousand gates per circuit.

4.3 Signed facts

Requiring the verifier to hold the current root is a strong assumption. In the signed mode, the attribute authority instead signs the 54-byte message `zk-pdp-root-v0` $\|$ root $\|$ epoch, with the epoch as eight big-endian bytes. The root becomes a private input, and the verifier pins only the authority's public key and the current epoch. One signature covers any number of opened records.

We support two signature schemes. For ECDSA over P-256, the authority signs the SHA-256 digest of the message. Barretenberg has a built-in gadget for this curve, and GordieHowe proves the signature in a second domain over the P-256 base field. Ed25519 matters because it is the key type that peer-to-peer trust stacks issue to their agents, but Noir has no built-in support for it. We wrote an Ed25519 verifier in Noir directly from RFC 8032, on top of noir-lang's `noir-bignum` library, with a SHA-512 implementation specialized to a fixed message length. It checks the cofactored equation $[8][S]B = [8]R + [8][k]A$ and decodes strictly: both coordinates must be below p, the x-coordinate supplied by the prover must match the sign bit and lie on the curve, S must be below the group order L, and public keys of small order are rejected. Point addition uses the complete twisted Edwards formulas, and scalar multiplication is a plain double-and-add chain over 253 bits, chosen for ease of audit rather than speed.

The verifier passes RFC 8032 test vectors 1 to 3, and each fails if the message is altered. Those vectors have messages of at most two bytes and exercise a single SHA-512 block; our 54-byte message needs two, which we test with signatures produced by Python's `cryptography` library. The circuit also rejects a wrong root, epoch or key; a modified S, $S + L$, $S = L$, and an R taken from another signature; and keys encoded with $y = p$, the identity, or a point of order two. The code has not been audited.

The signed mode provides integrity. It is not a complete private credential system: it offers no unlinkability, no device binding, and no revocation finer than an epoch.

5 Three backends

5.1 Cedar inside a zkVM

The zkVM guest is the published `cedar-policy` crate, version 4.14.1, in a thin wrapper. The policy set is an input rather than part of the program, so the program's identity, which the

verifier pins, does not change when the policy does. The port replaces one dependency, `stacker`, which relies on memory-mapping calls the guest lacks; Cedar's own source is unchanged. On all 83 requests in our corpus, the guest reproduces both the native evaluator's decision and its set of determining policies.

The guest reads the same two committed records as the circuits. It recomputes both leaves from the entities' JSON, checks their Merkle paths against the root, and gives Cedar only those two entities, with their closed ancestors as parents. It hashes the schema without parsing it, so for a request outside the schema it returns a proved denial where the circuits return no proof. Section 6.2 also measures a variant that parses the full entity store.

5.2 Compiling Cedar to circuits

The compiler starts from the terms Cedar's symbolic compiler produces, one per policy and per combination of principal type, action and resource type. For TinyTodo, these terms correspond one-for-one to the expressions in the hand-written circuit we use as a baseline. One policy, for instance, reduces to:

```
or(set.member(getattr["readers"](attrs[List](resource)),
              ancs[User, Team](principal)),
   set.member(getattr["editors"](attrs[List](resource)),
              ancs[User, Team](principal)))
```

The compiler emits one Noir function per type combination. Each function reads attribute and ancestor slots from the opened records. The circuit as a whole opens the records against the root, selects the function matching the request's types, and computes whether some `permit` holds and no `forbid` does. Hashing up to the root is shared across functions, and strings are compared by their hashes.

The compiler does not yet support extension values such as dates and IP addresses, integer arithmetic, entity tags, the `like` operator, or entities absent from the store. Of the terms in the upstream sample policy sets, 98.2 percent translate, and 5 of 7 whole sets compile (Section 6.3).

5.3 Targeting a machine-checked proof system

GordieHowe's language is embedded in C++, and rebuilding a C++ program for every policy change would be slow. The compiler therefore also emits each function as text in a small expression format, which a fixed C++ program parses at run time to build the relation. A policy change needs no C++ build, only a compilation of the relation, which takes 3.9 to 6.8 s. The resulting relation states the same logic and opens the same records as the Noir circuit. A hand-written TinyTodo relation serves as a baseline.

5.4 Toward a machine-checked chain

Because both Cedar's model and GordieHowe live in Lean, one can ask whether an accepted proof could be connected, by a single machine-checked argument, to the statement that Cedar's authorizer returns this decision on the committed facts. We know of no earlier attempt to connect a policy semantics to a proof system this way. Such a chain has six links:

A. *Cedar's authorizer agrees with evaluation of the symbolic terms.* SymCert proves this for the Lean version of the symbolic compiler [25]. Our compiler reads terms from the Rust crate, version 0.8.1, which the theorem does not cover; the chain should use the Lean compiler's terms.

B. *A witness satisfying our relation makes its decision bit equal to term evaluation over the committed facts.* Open.

C. *Record openings bind the facts*, under collision resistance of the hash. Open.

D. *The realized program implies the source program.* GordieHowe checks this per program with Lean's kernel.

E. *The argument is sound and zero-knowledge* whenever the source program is complete and round-by-round knowledge sound. GordieHowe proves this [23].

F. *Our source program is round-by-round knowledge sound.* Open, and substantial: GordieHowe's one end-to-end example required about 47 thousand lines of Lean, and we found no Lean specification of a SHA-256 gadget.

For TinyTodo we have attempted link D. A direct export of the program is 115 MB of Lean, most of it one shared expression printed as a tree; committing intermediate bits as witness values reduces it to 14 MB without slowing proving. The certificate's build exhausted our machine's memory before completing, so Lean's kernel has not accepted our program and we claim no machine-checked guarantee for it. Even a complete chain would leave some trust in place: that the C++ prover and verifier implement the Lean model, that the export records the program actually proved, and that the hash specifications match the standards.

6 Evaluation

We organize the evaluation around five questions:

Q1. What does one decision cost on each backend? (Section 6.1)

Q2. Where does that cost come from? (Section 6.2)

Q3. How much of Cedar does the compiler cover, and at what cost? (Section 6.3)

Q4. What do signed facts add? (Section 6.4)

Q5. Is proving practical in a browser? (Section 6.5)

Setup. All measurements use one Apple M1 Max laptop (10 cores, 64 GB, macOS 27.2). The corpus is TinyTodo and the upstream Cedar sample policy sets, each with an entity store and requests, and the native Cedar evaluator is the oracle for every decision. Noir figures are medians of five proofs of one request per set, with verification the median of 20 runs. GordieHowe figures are medians over every request in a set, five runs each. The zkVM proof is a single run and the Ed25519 figures are three runs. Appendix A lists tool versions.

Table 1: One TinyTodo decision. All three backends open the same two records from the same root.

	zkVM	Noir, compiled	GordieHowe, compiled
Size	5.38 M cycles	431,573 gates	n/a
Proving	768 s (one run)	2.08 s	42.7 ms
Threads	all cores (CPU prover)	10	1
Verification	126 ms	15 ms	14.6 ms
Proof	2.24 MB	14.7 KB	124.7 KB
Peak memory	12.4 GB	0.99 GB	not measured per proof
Per policy change	nothing	compile, 1.0 s key	3.9 s compile
Cedar coverage	all*	5 of 7 sets	5 of 7 sets

*For policies that read only the opened records.

6.1 One decision, three backends

Table 1 compares one TinyTodo decision across the three backends, each opening the same two records from the same root. Proving time spans roughly four orders of magnitude between the zkVM and GordieHowe, and proof size two orders of magnitude between the zkVM and Noir. Verification is fast everywhere.

The comparison is not exact, so we report orders of magnitude rather than ratios. The zkVM's cycle count is the median over 19 requests; the proved request took 5.61 million cycles, and three further proofs of it took 775 to 783 s. Noir's times include process start-up, while GordieHowe's do not, because its compiled system is already resident in memory. The zkVM accepts strings of any length, whereas the circuits refuse strings longer than their capacities and requests outside the schema. The zkVM proved on the CPU because RISC Zero 3.0.6 does not use the GPU for its main circuit on Apple silicon; a GPU or a server would change its column substantially. Its receipt is a composite receipt, which reveals the execution length; we did not measure succinct or Groth16 receipts.

The hand-written TinyTodo relations give the compiler's overhead. By hand, TinyTodo is 367,960 gates and proves in 1.85 s with Noir, and proves in 37.4 ms with a 115.8 KB proof on GordieHowe. The compiled versions are about 17 percent larger in gates on Noir and about 14 percent slower to prove on GordieHowe. The difference comes from longer string capacities and from slots that every generated circuit carries whether or not a given policy reads them.

6.2 Where the cost comes from

The zkVM's cost is almost entirely the cost of getting data into Cedar. Over the full 43-entity TinyTodo store, the full-store guest uses 23.3 million cycles per decision: 50 percent to parse the schema, 40 percent to parse the entities and compute their closure, and 9 percent to parse the policy set. Authorization itself takes 0.3 percent. Each entity costs about 215 thousand cycles, and the total grows linearly to 2.2 billion cycles at 10,000 entities. Across the whole corpus, authorization never exceeds 3.8 percent of cycles in this guest, or 5.5 percent in a variant that omits the schema.

Opening only the two records a decision needs removes most of this. The two-record guest takes a median of 5.38 million cycles, of which Cedar constructing its entities accounts for 3.08 million and parsing the policy set for 2.00 million; opening the records and authorizing take 0.13 and 0.07 million. The remaining levers are clear: supply only the entities a request can reach,

Table 2: Cost of each compiled policy set. Noir proves in zero-knowledge mode with 10 threads; its proofs are 14,656 bytes and verify in 15 ms for every set. GordieHowe uses one thread.

		Noir			GordieHowe			
Set	Requests	Gates	Prove	Memory	Prove	Verify	Proof	Compile
TinyTodo	19	431,573	2.08 s	0.99 GB	43 ms	15 ms	125 KB	3.9 s
GitHub example	7	609,470	2.94 s	1.25 GB	60 ms	21 ms	150 KB	5.3 s
Document cloud	5	625,162	3.03 s	1.17 GB	63 ms	25 ms	157 KB	5.5 s
Sales orgs	3	755,144	3.59 s	1.62 GB	84 ms	33 ms	160 KB	6.3 s
Hotel chains	6	836,248	3.90 s	2.09 GB	115 ms	43 ms	163 KB	6.8 s

Table 3: Hand-written TinyTodo with and without a signed root. On Noir, witness generation is a separate step before proving.

Backend	Authority	Size	Witness	Prove	Verify	Proof
Noir	none	367,960 gates	0.25 s	1.85 s	15 ms	14.7 KB
Noir	P-256	415,020 gates	0.26 s	2.07 s	15 ms	14.7 KB
Noir	Ed25519 (3 runs)	801,516 gates	5.45 s	4.12 s	18 ms	14.7 KB
GordieHowe	none			37.4 ms	13 ms	116 KB
GordieHowe	P-256			72.3 ms	33 ms	273 KB
zkVM	none	5.38 M cycles				
zkVM	P-256	5.58 M cycles				

omit the schema when the context holds no extension values, and pass policies and entities in a pre-parsed binary form. The only comparable figure we found elsewhere is a vendor's report that witness deserialization was half of proving cost at one stage of a client's guest.

The circuits show the same pattern in a different currency. In the hand-written TinyTodo circuit, we estimate the policy logic at a few hundred gates. Nearly everything else is SHA-256: the request hash, two identifier hashes, two leaves and two Merkle paths.

6.3 Compiler coverage

Five of the seven upstream sample policy sets compile: TinyTodo, the GitHub example, document cloud, sales organizations and hotel chains. The other two need unsupported features: `streaming_service` uses dates and integer arithmetic, and `tags_n_roles` uses entity tags.

On all 40 requests across the five compiled sets, both the Noir and GordieHowe backends accept the oracle's decision and reject its opposite. The upstream sets carry only 3 to 19 requests each, so this is a limited test of agreement. Table 2 reports the cost of each set.

Circuit size follows what a decision opens, not how many policies a set contains. Each opened entity costs roughly 170 thousand gates and each opened set some tens of thousands, while adding 100 policies to TinyTodo grew an earlier version of its circuit by only about 11 thousand gates. The policy is cheap; the facts are what one pays for.

6.4 The cost of signed facts

Table 3 adds an attribute authority's signature to the hand-written TinyTodo relation.

Table 4: Proving in Chrome 156 on the laptop. Noir uses `noir_js` and `bb.js` compiled to WebAssembly with 10 threads (medians of five or more runs). GordieHowe is compiled with Emscripten and runs on one thread in a Web Worker.

Backend	Circuit	Witness	Prove (browser)	Prove (native)
Noir	hand-written TinyTodo	0.14 s	3.8 s	1.85 s
Noir	hand-written TinyTodo, P-256	0.14 s	4.4 s	2.07 s
Noir	generated TinyTodo	0.16 s	4.5 s	2.08 s
Noir	generated GitHub example	0.30 s	6.2 s	2.94 s
Noir	generated document cloud	0.28 s	6.2 s	3.03 s
Noir	generated sales orgs	0.92 s	7.3 s	3.59 s
Noir	generated hotel chains	2.73 s	7.9 s	3.90 s
GordieHowe	generated TinyTodo, first 30 proofs		4.9–6.0 s	42.7 ms
GordieHowe	generated TinyTodo, next 15 proofs		≈1.2 s	42.7 ms

With Barretenberg's built-in gadget, a P-256 signature adds 47,060 gates. Ed25519, built from emulated field arithmetic, adds 433,556. On its own our Ed25519 check is 436,722 gates and proves in about 2.3 s. A third-party implementation reports about 228 thousand gates using windowed scalar multiplication, which suggests that about half of our verifier's cost can be optimized away. Ed25519 also has a less obvious cost: Noir's witness generation takes 5.45 s, longer than the proof itself, and peak memory rises to 2.0 GB from 0.73 GB with P-256. We suspect the big-number library's helper functions, which run in Noir's interpreter, but have not profiled this step.

On GordieHowe, a P-256 signature nearly doubles proving time and more than doubles proof size, because the signature requires a second field domain. In the zkVM it costs 198 thousand cycles using RISC Zero's precompile, against 69 thousand for authorization; those precompiles are documented as not constant-time, which matters if the signed data is meant to be private.

On all three backends, then, the signature costs far more than the policy but little relative to the rest of the proof. Prezta makes a related observation: on its heaviest policy, RSA verification and regular expressions together took about 92 percent of user cycles [28]. Bormann and Lehmann note that practical credential systems achieve their efficiency by fixing the policy, the format and the signature scheme in advance [3]. Keeping the policy as data is the opposite choice, and it has a real cost relative to specialized systems [15, 23].

6.5 Proving in a browser

Client-side proving matters only if it works where clients run. Table 4 reports proving in Chrome 156 on the same laptop.

Noir is consistently about twice as slow in the browser as natively, across every circuit. Individual runs occasionally stalled for up to 45 s, and we excluded a second pass in which proofs degraded to as much as 265 s for reasons we could not determine. Verification in the page takes 1.6 to 3.4 s, but only because the JavaScript verifier rederives the verification key on every call; a verifier that holds the key avoids this.

GordieHowe compiles to WebAssembly without source changes, using upstream's portable field arithmetic in place of carry-less multiplication instructions. Under Node 22 it proves in about 1.02 s and verifies in 0.34 s after a 14.6 s compile. In Chrome, compilation takes 71 s,

and the first 30 proofs took about 5 s each before the next 15 dropped to about 1.2 s, close to the Node figure. A later WebAssembly compilation tier in the browser may explain the change. These GordieHowe figures are preliminary, from 45 proofs of one policy. For neither backend have we measured threading, memory, or a phone.

7 Security evaluation

7.1 Adversarial witnesses

Differential testing exercises only honest witnesses, so it cannot detect a constraint missing from a circuit [27]. We therefore attack the circuits directly, in four ways.

Changed inputs (Noir). Each attack alters one fact, slot or public value and attempts to prove. Of 3,880 attacks against the five compiled circuits, 3,031 are rejected. The other 849 are accepted with the same decision and the same public inputs, because they alter a slot the selected function never reads or change nothing of substance. No attack produces an accepted proof of the opposite decision.

Changed witness values (Noir). We altered single witness values 366 times across three compiled circuits, proving and verifying each with Barretenberg. No change to a public input was accepted. Sixty-eight changes were accepted with all public inputs unchanged, and each matches a pattern our classifier checks explicitly. Sixty-two are the inverse hint of an is-zero check whose operand is zero, a value the constraints deliberately leave free. The other six are SHA-256 compression outputs; in a targeted test, every tampered proof was either byte-identical to the honest, deterministic proof or rejected, which indicates that the tampered value never reaches the proof.

GordieHowe. We ran 2,190 attacks against the five compiled sets and 236 against the signed hand-written relation, each with an honest control through the same path. No decision-changing attack was accepted. On the compiled sets, 240 attacks were accepted with an unchanged decision, all on slots the selected function does not read. Every rejection came from the verifier rather than from a crash in the prover.

Ed25519. All of the malformed inputs listed in Section 4.3 are rejected. We have not run witness-level attacks against this circuit.

The tests found one real gap. In the first version of the circuits, string padding was unconstrained, including in the public nonce, so one request admitted many encodings of its public inputs. This could not change a decision, and the fixed-layout hashing of Section 4.2 removes it by placing the padding inside the hash.

This evidence has limits. The attacks mostly change one value at a time, so a bug that needs several coordinated changes would go unnoticed. Noir's checker for unconstrained values is a connectivity heuristic, and it raises a warning inside the SHA-256 library we use, which we have not audited. Noir 1.0.0's own security policy does not yet recommend it for production use.

Table 5: Zero-knowledge status of each measured configuration.

Configuration	Status
RISC Zero, composite and succinct receipts	Unproven. The vendor states that no mathematical argument has been written. Haböck and Kindi report gaps they say have been patched, with no known attack [12]. Composite receipts reveal execution length.
RISC Zero, Groth16 wrapper	Plausibly zero-knowledge against the verifier, by reasoning from Groth16, but not claimed by the vendor. Its documentation says this prover runs only on x86, though one benchmark reports it on Apple silicon [29]. Not measured here.
Noir with UltraHonk, ZK mode	Designed to be zero-knowledge; we found no published proof. A critical flaw allowed forged zero-knowledge proofs before version 5.0.0.
GordieHowe	Proved in Lean for valid realized programs, under stated assumptions [23]. The authors also report gaps in published analyses of the Ligero protocol. For our program the source condition is not shown and the certificate is incomplete (Section 5.4).

7.2 Privacy

A succinct proof is not automatically a zero-knowledge one, and the backends differ sharply here (Table 5).

For the programs we measured, zero knowledge is therefore unproven on all three backends. GordieHowe is closest to a guarantee, but its theorems apply to our program only once the certificate checks and the source condition is established. Some leakage is intended: the decision, the hashes, and in the unsigned mode the root are public. A Merkle path reveals an entity's position in the store, so the store's ordering must not encode anything sensitive, and proving time and proof size can vary with the input, which we have not mitigated.

8 Discussion

The cost is in the facts. On every backend, evaluating the policy is the cheapest part of proving a decision. In the zkVM the cost is parsing; in the circuits it is SHA-256. We chose SHA-256 so that one published root could serve every backend, including the zkVM through its precompile. A hash designed for circuits would shrink the circuits considerably, but would either give up the shared root or force the zkVM to compute the same hash without a precompile. We have not measured that trade-off, and it is probably the most consequential open design choice.

Is it fast enough? Natively, GordieHowe proves the five compiled sets in 43 to 115 ms and Noir in 2 to 4 s. In a browser, the compiled Noir circuits take 4.5 to 7.9 s. Seconds are acceptable for a login or a grant that lasts a session, but not for a check on every API call. The setting that matters most for client-side proving, a phone, is one we have not measured, and for hotel chains Noir's 2.1 GB peak already exceeds what a phone browser allows.

Policy as data. The backends also differ in what a policy change costs. The zkVM needs nothing, since the policy is an input to a fixed program. Noir needs a compilation and 1 to 2 s to derive a verification key, and GordieHowe needs 3.9 to 6.8 s to compile a relation. These are per-policy, not per-request, costs. A verifier must still learn which key belongs to which policy. A zkVM proof that a given policy compiled to a given circuit would answer this, and we leave it to future work.

Peer-to-peer trust stacks. Systems that give every agent an Ed25519 key can already use such a key as an attribute authority in our signed mode. They cannot yet present their existing credentials directly, because our circuit verifies Ed25519 over our own 54-byte message rather than parsing a W3C credential.

Limitations. All measurements come from one laptop and one browser. We study one policy language, TinyTodo is small, and the upstream sets carry only 3 to 19 requests each. Our term-to-circuit translation and GordieHowe's run-time interpreter are unverified, and the Rust symbolic compiler they read from is not covered by SymCert's theorems. The compiler excludes real Cedar features, and the adversarial tests mostly change one value at a time. Links B, C and F of the Lean chain are open, and link D is incomplete for our program. We have not tested an unmodified AuthZEN enforcement point against these responses.

9 Related work

Proof-carrying authorization. The line of work begun by Appel and Felten and continued by Bauer, SD3 and Evidence-Based Audit [1, 2, 13, 26] proves authorization from premises the verifier can see, reasoning from positive evidence. We add succinct proofs, hidden premises, and proofs of non-monotonic outcomes, such as a `forbid` that does not fire, over a committed, closed world.

Zero-knowledge policy evaluation. Prezta [28] compiles each XACML policy into its own RISC Zero program. It proves in tens of seconds with proofs of 238 to 250 KB, makes no zero-knowledge claim, and has no committed fact store. Our zkVM backend differs in treating the policy as data and in reading committed records. Newton runs Rego in SP1 within a challenge protocol, a proposal called Warrant describes Cedar in a zkVM but is unbuilt, and ERC-8354 [9] carries policy verdicts proved with Noir and UltraHonk while arguing against compiling policies.

Policy compilers for circuits. Cinderella [7], ZK-TPL [19], zkGen [16] and Circuitree [11] compile smaller policy languages to circuits. CirC [20] lowers a single intermediate representation to both SMT and constraint systems, the same shape as our front end. We know of no compiler built on a verified symbolic compiler's terms, and none for Cedar.

Trusting the circuit. vkproof [17] and Ron and Monperrus [22] bind a verification key or a binary to its source. GordieHowe's per-program certificates answer a different question: whether the compiled program is faithful to its source.

Benchmarks. Sheybani et al. compare proof frameworks on generic workloads [24], and Yang et al. benchmark zkVMs without a circuit baseline [29]. We know of no comparison of a zkVM and circuits on the same policy language.

10 Conclusion

We proved Cedar authorization decisions on three kinds of proof system, each reading the same committed facts. The zkVM runs all of Cedar and absorbs policy changes for free, but one decision takes 13 minutes to prove on a laptop CPU. Compiled circuits cover five of seven sample policy sets and prove in seconds, including in a browser. On GordieHowe, the same compiled policies prove in tens of milliseconds, on a proof system whose theorems are checked in Lean. On all three, the cost lies in parsing or hashing the facts rather than in the policy. The most important remaining work is to complete the Lean certificate, test an unmodified AuthZEN enforcement point, measure proving on a phone, and replace single-value attacks with a proof.

Acknowledgments

We used Claude (Anthropic) for literature search, code and drafting.

A Artifact

Toolchains. RISC Zero 3.0.6 with Rust toolchain r0.1.97.0; `cedar-policy` 4.14.1 and `cedar-policy-symcc` 0.8.1; Noir 1.0.0, Barretenberg 6.0.0, `noir-lang/sha256` v0.3.0 and `noir-bignum` v0.10.0; GordieHowe at commit 4837770, built with clang 20.1.6; Emscripten 6.0.13; Chrome 156 with `noir_js` 1.0.0 and `bb.js` 6.0.0; Node 22. Upstream GordieHowe supports x86-64 Linux; we built it on Apple silicon with three small patches and clang 20 in place of the pinned clang 21, and did not run its test suite.

Oracle. Two of the upstream entity stores do not conform to their schemas; for those, the native evaluator ran without a schema.

Results. The table lists the files in the repository's `results/` directory behind each measurement.

Measurement	Files in `results/`
zkVM, full store	`E1-exec-2026-11-04`, `E1-prove-noschema-2026-11-05`
zkVM, two records	`E1b-exec-records-2026-11-05`, `E1b-prove-records-2026-11-05`
Hand-written Noir	`E2-tinytodo-2026-11-04`
Noir in the browser	`E2-E4-browser-*`
Signed facts, P-256	`E3-tinytodo-signed-2026-11-05`, `E3-exec-signed-2026-11-05`
Compiler coverage	`E4-coverage-*`, `E4-compiled-v1-2026-11-05`
GordieHowe	`E8-tinytodo-gordiehowe-2026-11-04`, `E11-policy-v1-smoke-2026-11-05`
Adversarial witnesses	`E9-input-attacks-v1-*`, `E9-witness-*-v1-*`
GordieHowe, Node and browser	`E12-gordiehowe-node-*`, `E12-gordiehowe-browser-*`
Signed facts, Ed25519	`E13-ed25519-signed-tinytodo-2026-11-05`
Final timing pass	`T-*-2026-11-05`

References

[1] Andrew W. Appel and Edward W. Felten. Proof-carrying authentication. In *Proceedings of the 6th ACM conference on Computer and communications security*, pages 52–62, 1999. doi: 10.1145/319709.319718. URL https://doi.org/10.1145/319709.319718.

[2] Lujo Bauer, Michael A. Schneider, and Edward W. Felten. A Proof-Carrying Authorization System. Technical Report TR-638-01, Princeton University, 2001. URL http://users.ece.cmu.edu/~lbauer/papers/pcaprototr.pdf.

[3] Christian Bormann and Anja Lehmann. SoK: Anonymous Credentials for Digital Identity Wallets. In *Lecture Notes in Computer Science*, pages 3–25, 2026. doi: 10.1007/978-3-032-19567-8_1. URL https://doi.org/10.1007/978-3-032-19567-8_1.

[4] Jeremy Bruestle, Paul Gafni, and RISC Zero Team. RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity. Draft, August 11, 2023, 2023. URL https://dev.risczero.com/proof-system-in-detail.pdf.

[5] Jalen Chuang, Alex Seto, Nicolas Berrios, Stephan van Schaik, Christina Garman, and Daniel Genkin. TEE.fail: Breaking Trusted Execution Environments via DDR5 Memory Bus Interposition. In *47th IEEE Symposium on Security and Privacy*, 2026. URL https://tee.fail.

[6] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):670–697, 2024. doi: 10.1145/3649835.

[7] Antoine Delignat-Lavaud, Cedric Fournet, Markulf Kohlweiss, and Bryan Parno. Cinderella: Turning Shabby X.509 Certificates into Elegant Anonymous Credentials with the Magic of Verifiable Computation. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 235–254, 2016. doi: 10.1109/sp.2016.22. URL https://doi.org/10.1109/sp.2016.22.

[8] Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, John Kastner, Anwar Mamat, Matt McCutchen, Neha Rungta, Bhakti Shah, Emina Torlak, and Andrew Wells. How We Built Cedar: A Verification-Guided Approach. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 351–357, 2024. doi: 10.1145/3663529.3663854.

[9] Muhammad Zidan Fatonie, Faisal Firdani, and Maulana Asykari Muhammad. ERC-8354: Confidential agent policy verdicts. Ethereum ERC, Draft, created 2026-07-16, 2026. URL https://github.com/ethereum/ERCs/blob/master/ERCS/erc-8354.md.

[10] O. Gazitt, D. Brossard, and A. Tulshibagwale. Authorization API 1.0. OpenID Foundation, Final Specification, 2026. URL https://openid.net/specs/authorization-api-1_0.html.

[11] Tom Godden, Ruben De Smet, Christophe Debruyne, Thibaut Vandervelden, Kris Steenhaut, and An Braeken. Circuitree: A Datalog Reasoner in Zero-Knowledge. *IEEE Access*, 2022. doi: 10.1109/access.2022.3153366.

[12] Ulrich Haböck and Al Kindi. A note on adding zero-knowledge to STARKs. Cryptology ePrint Archive, Paper 2024/1037, revised 2025-02-22, 2024. URL https://eprint.iacr.org/2024/1037.

[13] T. Jim. SD3: a trust management system with certified evaluation. In *Proceedings 2001 IEEE Symposium on Security and Privacy*, pages 106–115, 2001. doi: 10.1109/SECPRI.2001.924291.

[14] Xisen Jin, Michael Duan, Qin Lin, Aaron Chan, Zhenglun Chen, Junyi Du, and Xiang Ren. Proof-of-Guardrail in AI Agents and What (Not) to Trust from It, 2026.

[15] Darya Kaviani and Srinath Setty. Vega: Low-Latency Zero-Knowledge Proofs over Existing Credentials. In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 1951–1969, 2026. doi: 10.1109/sp63933.2026.00182. URL https://doi.org/10.1109/sp63933.2026.00182.

[16] Jan Lauinger, Jens Ernstberger, and Sebastian Steinhorst. zkGen: Policy-to-Circuit Transpiler. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 620–624, 2024. doi: 10.1109/icbc59979.2024.10634440. URL https://doi.org/10.1109/icbc59979.2024.10634440.

[17] Antonio Mejias Gil and Xueqin Zhao. vkproof: Succinct verification of indexed verifying keys using modular compilation and polynomial fingerprinting. Cryptology ePrint Archive, Paper 2026/581, 2026. URL https://eprint.iacr.org/2026/581.

[18] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: Bringing Key Transparency to End Users. In *24th USENIX Security Symposium*, pages 383–398, 2015. URL https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/melara.

[19] Stefan More, Sebastian Ramacher, Lukas Alber, and Marco Herzl. Extending Expressive Access Policies with Privacy Features. In *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 574–581, 2022. doi: 10.1109/trustcom56396.2022.00084. URL https://doi.org/10.1109/trustcom56396.2022.00084.

[20] Alex Ozdemir, Fraser Brown, and Riad S. Wahby. CirC: Compiler infrastructure for proof systems, software verification, and more. In *2022 IEEE Symposium on Security and Privacy (SP)*, 2022. doi: 10.1109/SP46214.2022.9833782.

[21] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. Zanzibar: Google's Consistent, Global Authorization System. In *2019 USENIX Annual Technical Conference*, pages 33–46, 2019. URL https://www.usenix.org/conference/atc19/presentation/pang.

[22] Javier Ron and Martin Monperrus. Verifiable Provenance of Software Artifacts with Zero-Knowledge Compilation, 2026. URL https://arxiv.org/abs/2602.11887.

[23] Chad Sharp and Paul Grubbs. Faster and Smaller Anonymous Credentials from Standardized Signatures. Cryptology ePrint Archive, Paper 2026/2615, 2026. URL https://eprint.iacr.org/2026/2615.

[24] Nojan Sheybani, Anees Ahmed, Michel Kinsy, and Farinaz Koushanfar. Zero-Knowledge Proof Frameworks: A Systematic Survey, 2025.

[25] Emina Torlak. SymCert: Verifying SMT-based policy analyses. In *Formal Methods in Computer-Aided Design*, 2026. URL https://www.amazon.science/publications/symcert-verifying-smt-based-policy-analyses.

[26] Jeffrey A. Vaughan, Limin Jia, Karl Mazurak, and Steve Zdancewic. Evidence-Based Audit. In *2008 21st IEEE Computer Security Foundations Symposium*, pages 177–191, 2008. doi: 10.1109/CSF.2008.24.

[27] Sebastian Watzinger, Christoph Hochrainer, Valentin Wüstholz, and Maria Christakis. Lie to Me: Finding Bugs in ZK DSL Toolchains with Adversarial Witness Injection, 2026.

[28] Zhongjing Wei, Osaid Muhammad Ameer, Yupeng Zhang, and Nikita Borisov. Prezta: Provable Remote Execution of Zero-Trust Authorization using SNARKs. In *35th USENIX Security Symposium (USENIX Security 26)*, pages 5811–5830. USENIX Association, August 2026. ISBN 978-1-939133-58-8. URL https://www.usenix.org/conference/usenixsecurity26/presentation/wei-zhongjing.

[29] Yunbo Yang, Yuejia Cheng, Haibo Tang, Guomin Yang, Bingsheng Zhang, and Kui Ren. SoK: Understanding zkVM: From research to practice. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*, pages 852–868, 2026. doi: 10.1145/3779208.3805964.